

Experiment Title

Implementation of Left Recursion Elimination in Context-Free Grammar Using C++

Objectives

The objectives of this experiment are:

- To understand the concept of left recursion in context-free grammars (CFGs).
- To identify immediate left-recursive productions in a grammar.
- To eliminate immediate left recursion using the standard transformation technique.
- To implement the left recursion elimination algorithm using the C++ programming language.
- To generate an equivalent grammar that is suitable for top-down parsing techniques such as Recursive Descent and LL(1) parsing.

Introduction

A compiler is system software that translates a high-level programming language into machine code. One of the major phases of compiler design is **syntax analysis (parsing)**, where the compiler verifies whether the source code follows the grammatical rules of the programming language.

Parsing techniques are generally divided into **top-down** and **bottom-up** approaches. Top-down parsers, such as **Recursive Descent Parser** and **LL(1) Parser**, cannot correctly parse grammars containing **left recursion**, because they may enter infinite recursive calls.

A grammar is said to contain **left recursion** when a non-terminal appears as the leftmost symbol on the right-hand side of one of its own productions.

For example,

$$A \rightarrow Aa \mid b$$

In this grammar, the production $A \rightarrow Aa$ is left recursive because the production begins with the same non-terminal A .

To make such grammars suitable for top-down parsing, left recursion must be eliminated. The standard transformation for immediate left recursion is:

Original Grammar:

$$A \rightarrow A\alpha \mid \beta$$

Equivalent Grammar after eliminating left recursion:

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \epsilon$$

Where,

- α represents the recursive part,
- β represents the non-recursive part,
- ϵ (**epsilon**) denotes the empty string.

In this experiment, the elimination of immediate left recursion is implemented using C++, where the symbol # is used to represent ϵ (**epsilon**).

Discussion

This experiment focuses on eliminating **immediate left recursion** from a context-free grammar. The developed C++ program accepts a parent non-terminal, the number of productions, and each production from the user. It then analyzes every production to determine whether it is left recursive.

If a production begins with the parent non-terminal, it is identified as a left-recursive production. The program extracts the remaining portion of that production (α). Productions that do not begin with the parent non-terminal are treated as non-recursive productions (β).

After separating the recursive and non-recursive parts, the program constructs a new equivalent grammar using the standard left recursion elimination rule. The transformed grammar preserves the original language while removing left recursion, making it suitable for top-down parsing.

The implementation also displays whether each production is left recursive or not before generating the final grammar.

Algorithm

1. Read the parent non-terminal from the user.
2. Read the total number of productions.
3. Store all production strings.
4. Examine each production individually.
5. If a production starts with the parent non-terminal:
 - Mark it as left recursive.
 - Store the remaining portion as α .
6. Otherwise:
 - Store the production as β .
7. Generate the new grammar:
$$A \rightarrow \beta A'$$
$$A' \rightarrow \alpha A' \mid \varepsilon$$
8. Display the transformed grammar.

Example:

Input Grammar

$$A \rightarrow Aa \mid b$$

The program identifies

Recursive production:

$$A \rightarrow Aa$$

Non-recursive production:

$$A \rightarrow b$$

After eliminating left recursion, the equivalent grammar becomes

$$A \rightarrow bA'$$

$$A' \rightarrow aA' \mid \varepsilon$$

Program Output:

Enter the Parent Non-Terminal : A

Enter the number of productions : 2

Enter Production 1 : Aa

Enter Production 2 : b

Production Rule :

A->Aa|b

Production 1 has left recursion.

Production 2 does not have left recursion.

Final Grammar (after removing left recursion):

$A \rightarrow bA'$

$A' \rightarrow aA' \mid \#$

Note: In this implementation, the symbol # represents the empty string ϵ (epsilon).

Observation

- The program correctly identifies immediate left-recursive productions.
- Recursive and non-recursive productions are separated successfully.
- The transformed grammar is equivalent to the original grammar.
- The resulting grammar is suitable for top-down parsing techniques such as LL(1) and Recursive Descent parsing.
- The current implementation eliminates **only immediate left recursion** and does not handle **indirect left recursion**.

Conclusion

This experiment successfully demonstrates the elimination of immediate left recursion from a context-free grammar using C++. The program correctly detects left-recursive productions, separates recursive and non-recursive components, and generates an equivalent grammar without changing the language recognized by the grammar. Eliminating left recursion is an essential preprocessing step in compiler construction because it enables efficient implementation of top-down parsers and prevents infinite recursive parsing.

References

1. Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman, *Compilers: Principles, Techniques, and Tools* (2nd Edition), Pearson Education.
2. Kenneth C. Loudon, *Compiler Construction: Principles and Practice*, Cengage Learning.
3. John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, Pearson.
4. Compiler Design Laboratory Manual, Department of Computer Science and Engineering, Bangladesh University of Business and Technology (BUBT).